

# Les bases de données relationnelles

## Les jointures

# Jointures

## Introduction

- Permet de combiner des données de plusieurs tables dans une seule requête.
- Utilisé pour exploiter les relations entre tables dans une base de données relationnelle.
- Évite la duplication des données et optimise les requêtes.

# Types de jointures

## INNER JOIN

- Récupère uniquement les enregistrements ayant des correspondances dans les deux tables.
- Syntaxe:

```
SELECT clients.nom, commandes.date  
FROM clients  
INNER JOIN commandes ON clients.id = commandes.client_id;
```

# Types de jointures

## LEFT JOIN (LEFT OUTER JOIN)

- Récupère tous les enregistrements de la table de gauche et ceux correspondants de la table de droite. Si aucune correspondance, NULL est retourné.
- Syntaxe:

```
SELECT clients.nom, commandes.date  
FROM clients  
LEFT JOIN commandes ON clients.id = commandes.client_id;
```

# Types de jointures

## RIGHT JOIN (RIGHT OUTER JOIN)

- Récupère tous les enregistrements de la table de droite et ceux correspondants de la table de gauche. Si aucune correspondance, NULL est retourné.
- Syntaxe:

```
SELECT clients.nom, commandes.date  
FROM clients  
RIGHT JOIN commandes ON clients.id = commandes.client_id;
```

# Types de jointures

## FULL JOIN (FULL OUTER JOIN)

- Récupère tous les enregistrements de chaque table, avec des valeurs NULL si aucune correspondance.
- MySQL ne supporte pas FULL OUTER JOIN directement, mais on peut le simuler avec UNION.
- Syntaxe:

```
SELECT clients.nom, commandes.date
FROM clients
LEFT JOIN commandes ON clients.id = commandes.client_id
UNION
SELECT clients.nom, commandes.date
FROM clients
RIGHT JOIN commandes ON clients.id = commandes.client_id;
```

# Types de jointures

## CROSS JOIN

- Produit un produit cartésien entre les deux tables (chaque ligne de la table A est combinée avec chaque ligne de la table B).
- Syntaxe:

```
SELECT clients.nom, commandes.date  
FROM clients  
CROSS JOIN commandes;
```

# Types de jointures

## SELF JOIN

- Jointure d'une table avec elle-même.
- Exemple : Trouver les employés supervisés par un autre employé
- Syntaxe:

```
SELECT A.nom AS employe, B.nom AS superviseur  
FROM employes A, employes B  
WHERE A.superviseur_id = B.id;
```

# Jointures

## Contraintes et bonnes pratiques

- Utiliser des alias pour rendre les requêtes plus lisibles.
- Optimiser avec des index sur les colonnes utilisées pour les jointures.
- Préférer les INNER JOIN lorsque possible pour de meilleures performances.
- Tester avec EXPLAIN pour analyser les performances des requêtes.

# Jointures

## Exercice pratique

1. Récupérer la liste des articles et leurs commandes (INNER JOIN).
2. Récupérer tous les articles avec leurs commandes (LEFT JOIN).
3. Trouver les commandes qui n'ont pas d'articles correspondants (RIGHT JOIN).